

Git, GitHub & Pull Request Complete Workflow Guide

A Comprehensive Standard Operating Procedure for Version Control, Branch Management, Pull Requests, Code Reviews, and Production Releases.

TARGET REPOSITORY

TheNexoraLab / Project-OhmSim

CORE BRANCHES

main (Production) • development (Integration)

FRAMEWORK STACK

Next.js 15, Prisma ORM, PostgreSQL (Neon)

RELEASE VERSION

v1.0.0 (August 2026 Edition)

00 Table of Contents

Chapter 1 — Git & Version Control Core Principles	Page 3
Chapter 2 — The Two-Tier Branch Architecture (main vs development)	Page 4
Chapter 3 — Complete Step-by-Step Feature Development Lifecycle	Page 5
Chapter 4 — Deep Dive: Anatomy of a Great Pull Request (PR)	Page 7
Chapter 5 — Merging Strategies Compared (Merge Commit vs Squash vs Rebase)	Page 8
Chapter 6 — Production Release & Promotion Workflow	Page 9
Chapter 7 — Emergency Playbook & Troubleshooting Scenarios	Page 10
Chapter 8 — Quick Reference Terminal Command Cheat Sheet	Page 11

■ Handbook Alignment

This document serves as the operational companion to the official **OhmSim Product Design & Development Handbook**. It strictly enforces the development rules, commit formatting, and QA release pipelines established by NEXORA Labs.

01 Git & Version Control Fundamentals

Git is a distributed version control system that records changes to files over time so that specific versions can be recalled, reviewed, and merged collaboratively without data loss.

The Four Environments in Git

Understanding how code moves across environments is essential before running any commands:



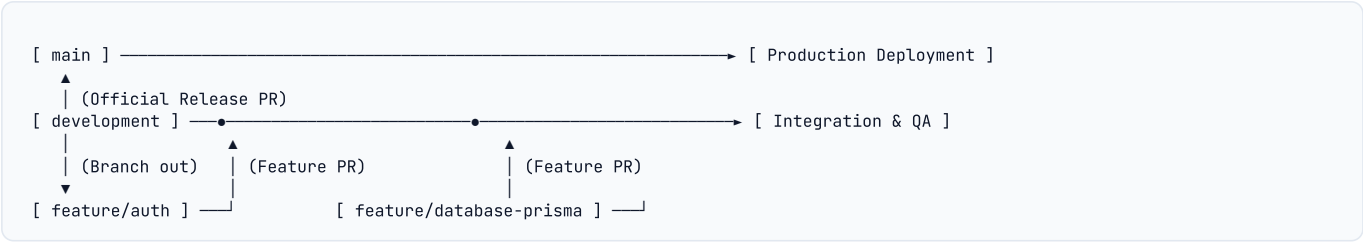
Environment	Location	Description & Purpose
Working Tree	Local Disk	The actual directory and files you edit in VS Code / IDE.
Staging Area	Memory / Index	The staging area where changes are prepared and gathered before taking a snapshot.
Local Repository	.git/ folder	Committed snapshots stored permanently in your local history database.
Remote Repository	GitHub (origin)	The cloud repository shared across team members, CI/CD pipelines, and Vercel.

What is a Branch?

In Git, a **branch** is simply a lightweight, movable pointer to a specific commit. Creating branches allows developers to isolate experimental code, develop new features, or fix bugs without affecting the stable application running on other branches.

02 The Two-Tier Branch Architecture

OhmSim uses a structured two-tier branching model to protect the production application while allowing continuous integration of new features.



Permanent vs Temporary Branches

Branch Type	Name Pattern	Lifetime	Purpose & Policy
PERMANENT	main	Forever	Production Branch: Always represents live, fully tested code. Direct commits are strictly prohibited. Automatically deploys to Vercel production.
PERMANENT	development	Forever	Integration Branch: Day-to-day meeting ground for all completed features. Tested in staging before merging into main.
TEMPORARY	feature/<name>	1 Task	Dedicated to building a specific feature (e.g. feature/database-prisma). Deleted immediately after merging into development.
TEMPORARY	fix/<issue-name>	1 Bugfix	Dedicated to fixing a specific bug. Merged into development via PR and deleted.
TEMPORARY	docs/<topic>	1 Task	Documentation updates and specifications. Merged and deleted.

⚠ The Cardinal Rule of OhmSim Development

NEVER commit application code directly to `main` or `development`. Always branch off `development` into a feature branch, test locally, and submit a Pull Request.

03 Step-by-Step Feature Development Lifecycle

Every single task in OhmSim follows this strict 8-step lifecycle:

1 Sync & Prepare Integration Baseline

Always ensure your local development branch is 100% updated before branching:

```
git switch development
git pull --ff-only origin development
```

2 Create Focused Feature Branch

Create a descriptive branch with the standard prefix:

```
git switch -c feature/database-prisma
```

3 Implement Code & Verify Locally

Write your TypeScript code, components, or APIs. Run linting and test builds before committing:

```
npm run lint
npm run build
```

4 Commit using Conventional Standards

Follow the OhmSim commit convention (feat:, fix:, docs:, refactor:, chore:):

```
git add .
git commit -m "feat: implement prisma schema with 12 core database models"
```

03 Feature Lifecycle (Steps 5 – 8)

5 Push Feature Branch to GitHub

Publish your feature branch to the remote repository:

```
git push -u origin feature/database-prisma
```

6 Open Pull Request on GitHub

Navigate to GitHub and click **"Compare & pull request"**:

- **Base Branch:** `development` **CRITICAL** (Do NOT target main directly).
- **Compare Branch:** `feature/database-prisma`
- Fill in the PR Title and Description summarizing what was implemented and tested.

7 Merge Pull Request & Delete Remote Branch

Once approved and all checks pass, click **"Squash and merge"** on GitHub, then click the purple **"Delete branch"** button to remove the remote tracking branch.

8 Sync Local Development & Delete Local Branch

Return to your local machine, pull the newly merged code into development, and delete the stale local branch:

```
git switch development
git pull --ff-only origin development
git branch -d feature/database-prisma
```

🎉 Clean State Achieved

Your local machine is now back in sync with GitHub, and ready to start the next feature task immediately!

04 Anatomy of a High-Quality Pull Request

A Pull Request is not just a merge button; it is a permanent engineering document and code review artifact. High-quality PRs make debugging and project maintenance effortless.

Recommended PR Description Template

```
## Summary of Changes
Implemented the PostgreSQL database models using Prisma ORM according to Chapter 10 of the Handbook.

## Key Features / Additions
- Added schema for User, Product, Category, Order, BOMProject, and UserAddress.
- Added database constraints and relations.
- Configured Neon connection pooling.

## Verification & Testing Performed
- [x] Ran npm run lint with 0 warnings/errors.
- [x] Ran npx prisma validate and verified relations.
- [x] Verified migrations in local environment.

## Screenshots / Artifacts (if UI change)
[Attached UI screenshots if applicable]
```

Pull Request Review Checklist

Review Area	What Reviewers Look For
Functionality	Does the code satisfy all business rules specified in Chapter 8?
TypeScript Types	Are all props, API responses, and database queries strictly typed? No 'any' types.
Security	Are server-side authorization and input validation present? No hard-coded secrets.
Cleanliness	Are debug logs (console.log), temporary mocks, and commented-out code removed?

05 Merging Strategies Compared

GitHub provides three distinct merge strategies. Each has specific trade-offs and designated use-cases in OhmSim.

Strategy	How It Operates	Commit History Graph	When to Use in OhmSim
Squash and merge RECOMMENDED FOR FEATURES	Combines all commits from the feature branch into 1 clean commit on the target branch.	 dev: (1 single tidy commit)	Feature & Bugfix PRs into development. Keeps the integration history readable and easy to revert.
Create a merge commit STANDARD RELEASE	Preserves all individual commits and adds a dedicated merge commit linking the branches.	 main: (Merge) dev: \●—●/	Release PRs from development into main. Formally marks production release milestones.
Rebase and merge LINEAR	Applies all commits one-by-one directly on top of the base branch without a merge commit.	 dev: (All commits in straight line)	Used when every single intermediate commit in the branch was carefully structured and squashed beforehand.

 **NEXORA Labs Recommendation**

- For `feature/*` → `development` : Use **Squash and merge**.
- For `development` → `main` : Use **Create a merge commit**.

06 Production Release & Promotion Workflow

When a set of features in `development` forms a stable milestone (such as MVP baseline or a full sprint goal), it is promoted to `main`.

1 Pre-Release Quality Audit

Ensure that all planned features in `development` are complete, all unit/integration tests pass, and no blocking P0/P1 bugs remain.

2 Open Production Release PR

Open a Pull Request with target `base: main` ← `compare: development`.

3 Merge via Merge Commit & Vercel Deployment

Merge PR into `main` using "Create a merge commit". Vercel automatically builds and deploys the production bundle.

4 Git Semantic Tagging

Tag the release version on `main` so production snapshots can always be recalled:

```
git switch main
git pull --ff-only origin main
git tag -a v1.0.0 -m "Release v1.0.0 - Baseline Project Setup & Documentation"
git push origin v1.0.0
```

07 Emergency Playbook & Troubleshooting

Scenario 1: Accidental Commits on development (Unpushed)

If you accidentally wrote and committed code directly on development before creating a feature branch:

```
# 1. Create and switch to the new feature branch (your commits come with you)
git switch -c feature/my-feature

# 2. Switch back to development and reset it to match origin
git switch development
git reset --hard origin/development

# 3. Switch back to your feature branch and continue working safely
git switch feature/my-feature
```

Scenario 2: Discarding Unwanted Local Changes

```
# Discard changes in a specific file:
git restore path/to/file.tsx

# Discard all unstaged changes across entire project:
git restore .

# Unstage a file added with 'git add':
git restore --staged path/to/file.tsx
```

Scenario 3: Resolving Merge Conflicts

1. Switch to your feature branch: `git switch feature/my-feature`
2. Merge latest development: `git merge development`
3. Open VS Code. Conflicts will show with standard merge markers. Choose *"Accept Incoming"* or *"Accept Current"*.
4. Test build: `npm run build`
5. Stage and commit the resolution: `git add . && git commit -m "fix: resolve merge conflicts with development"`
6. Push: `git push origin feature/my-feature`

08 Quick Reference Command Cheat Sheet

Task / Action	PowerShell Terminal Command
Check status & branch	<code>git status</code>
List local branches	<code>git branch</code>
List all (local + remote)	<code>git branch -a</code>
Switch branch	<code>git switch <branch-name></code>
Create & switch to branch	<code>git switch -c feature/<work-name></code>
Fetch updates from GitHub	<code>git fetch origin</code>
Pull latest development	<code>git pull --ff-only origin development</code>
Stage all changes	<code>git add .</code>
Commit with message	<code>git commit -m "feat: description"</code>
Push branch to GitHub	<code>git push -u origin feature/<work-name></code>
Delete merged local branch	<code>git branch -d feature/<work-name></code>
Force delete unmerged branch	<code>git branch -D feature/<work-name></code>
View one-line commit log	<code>git log --oneline --graph --decorate -n 15</code>

 NEXORA Labs Engineering Standard

Keep this handbook handy whenever pair-programming or submitting features. Adhering to these version control protocols guarantees that OhmSim remains clean, reliable, and production-ready at all times.